

ARKIV x PunaTech 2026

Starter Repo para principiantes

Guía práctica para crear, explicar y entregar un proyecto de IA usando Arkiv como capa de datos.

Objetivo: Que un participante pueda abrir el repo, entender el modelo de Arkiv, crear su primera entidad, consultar datos por atributos, preparar la demo y entregar sin perder puntos por documentación o estructura.

Dato clave	Valor
Track	Aplicaciones de IA sobre ARKIV como capa de datos
Fechas	28 al 30 de mayo de 2026
Cierre de entregas	30 de mayo de 2026, 2:00 PM, hora Argentina
Anuncio de ganadores	30 de mayo de 2026, 4:00 PM, hora Argentina
Premios	600, 450, 300 y 150 USDC
Entrega	Repo publico, demo desplegada, video, pitch en español y formulario oficial

Repositorio oficial: <https://github.com/Arkiv-Network/arkiv-puna-tech-hackathon>

1. Que es este repo

Este documento resume el starter repo educativo para participantes del hackathon ARKIV x PunaTech 2026. No es una regla oficial ni implica patrocinio. Es una base tecnica para construir rapido y explicar bien una demo.

El repo esta pensado para principiantes que necesitan una estructura clara: README, quickstart, checklist, modelos de datos, ejemplos TypeScript y una estrategia de entrega alineada con la rubrica.

Archivo o carpeta	Funcion
README.md	Explica el proyecto, el track, fechas, red, estructura y estrategia.
QUICKSTART.md	Guia paso a paso para crear y consultar la primera entidad.
SUBMISSION-CHECKLIST.md	Checklist antes de enviar el formulario oficial.
docs/01-mental-model-arkiv.md	Explica entidades, payload, atributos y expiracion.
docs/02-schema-design.md	Ayuda a disenar entidades y relaciones.
docs/03-build-ideas.md	Ideas de MVP validas para el track.
docs/04-scoring-strategy.md	Como maximizar puntos segun la rubrica.
docs/05-pitch-template.md	Pitch breve en espanol.
src/arkiv/	Codigo base para configurar, crear y consultar entidades.

2. Track y enfoque correcto

El track pide aplicaciones de IA que usen Arkiv como capa de datos. La diferencia entre una demo debil y una fuerte no es decir que se usa blockchain, sino demostrar que Arkiv organiza, consulta y controla datos reales del producto.

Ideas validas

- Memoria portable de agentes de IA controlada por el usuario.
- Auditoria de decisiones, prompts, outputs o acciones de modelos.
- Perfiles personales para IA con preferencias y permisos del usuario.
- Logs de agentes, historial de prompts, reputacion o trazabilidad de datos.
- Memoria de equipo para registrar decisiones durante el hackathon.

Criterio tecnico: Arkiv debe ser la capa de datos principal. No alcanza con guardar un JSON sin atributos. La app debe crear entidades, consultarlas por atributos, explicar ownership, usar expiresIn y separar datos por PROJECT_ATTRIBUTE.

3. Fechas, premios y entrega

Hito	Fecha
Inicio del hackathon	28 de mayo de 2026
Cierre de entregas	30 de mayo de 2026, 2:00 PM, hora Argentina
Anuncio de ganadores	30 de mayo de 2026, 4:00 PM, hora Argentina

Puesto	Premio
1er lugar	600 USDC
2do lugar	450 USDC
3er lugar	300 USDC
4to lugar	150 USDC

Entregables minimos

1. Repositorio publico en GitHub con licencia open source.
2. Demo funcional desplegada y conectada a Arkiv testnet.
3. Video publico de 2 a 3 minutos mostrando el flujo.
4. Pitch en español.
5. Formulario oficial completo: <https://forms.arkiv.network/punatech26>.
6. README con instalacion, stack, equipo, vertical elegida y enfoque tecnico.

KYC: El KYC no es necesario para participar, pero si para cobrar premio. Todos los integrantes del equipo ganador deben completarlo individualmente.

4. Quickstart tecnico

Objetivo: crear y consultar una entidad en Arkiv testnet usando TypeScript. Usar una wallet de testnet y no subir claves privadas a GitHub.

Requisitos

- Node.js 20+
- npm 10+
- Wallet EVM para testnet
- Fondos desde el faucet de Braga
- Private key solo de prueba local

```
npm install
cp .env.example .env
```

Editar .env:

```
PRIVATE_KEY=0xTU_PRIVATE_KEY_DE_TESTNET
PROJECT_SLUG=tu-equipio-arkiv-punatech26
```

Obtener gas de testnet:

```
https://braga.hoodi.arkiv.network/faucet/
```

Crear una entidad de ejemplo:

```
npm run arkiv:create-memory
```

Consultar entidades del proyecto:

```
npm run arkiv:query-memories
```

Revisar checklist antes de entregar:

```
npm run pre-submit
```

5. Red de prueba

Configuracion	Valor
Network ID	60138453102
RPC HTTP	https://braga.hoodi.arkiv.network/rpc
RPC WebSocket	wss://braga.hoodi.arkiv.network/rpc/ws
Bridge contract	0xB52b417A79c9dE21ffe221dF9a3821B7EaC60813
Faucet	https://braga.hoodi.arkiv.network/faucet/

6. Modelo mental de Arkiv

En Arkiv no conviene pensar primero en tablas. Conviene pensar en entidades.

Componente	Funcion
Payload	Contenido real: JSON, texto o bytes.
Atributos	Indices consultables. Todo filtro importante debe estar aca.
expiresIn	Tiempo de vida de la entidad, expresado en segundos.
\$owner	Wallet que controla actualizaciones o eliminaciones.
\$creator	Wallet que creo la entidad originalmente; sirve para atribucion.

Regla critica: No filtres por payload. Los atributos son el indice. Si un dato se va a buscar, filtrar u ordenar, debe ir como atributo tipado.

7. PROJECT_ATTRIBUTE

Arkiv es una base compartida. Todas las entidades de todos los proyectos pueden vivir en el mismo entorno. Por eso cada proyecto necesita un namespace unico.

```
export const PROJECT_ATTRIBUTE = {
  key: "project",
  value: "mi-equipo-arkiv-punatech26-7x9k",
} as const;
```

Buenas practicas:

- Usarlo en cada creacion de entidad.
- Usarlo en cada actualizacion.
- Usarlo en cada query.
- No usar valores genericos como demo, test o arkiv.
- Documentarlo en el README.

8. Esquema inicial recomendado

Para principiantes, el MVP mas defendible es una combinacion de AI Memory Wallet y Audit Trail minimo.

Entidad: ai_memory

```
attributes: [  
  { key: "project", value: PROJECT_SLUG },  
  { key: "type", value: "ai_memory" },  
  { key: "agentId", value: "demo-agent" },  
  { key: "userId", value: "demo-user" },  
  { key: "importance", value: 4 },  
  { key: "createdAt", value: Date.now() }  
]  
  
payload: {  
  "title": "Preferencia del usuario",  
  "content": "El usuario prefiere respuestas tecnicas y directas.",  
  "source": "chat",  
  "tags": ["preference", "memory", "ai"]  
}
```

Entidad: audit_log

```
attributes: [  
  { key: "project", value: PROJECT_SLUG },  
  { key: "type", value: "audit_log" },  
  { key: "agentId", value: "demo-agent" },  
  { key: "risk", value: "low" },  
  { key: "createdAt", value: Date.now() }  
]  
  
payload: {  
  "promptSummary": "Usuario pide recomendacion tecnica",  
  "outputSummary": "Agente responde con criterios y pasos",  
  "model": "demo-model",  
  "decision": "allowed"  
}
```

9. Como puntuar mejor

Criterio	Peso	Como subir puntos
Integracion con Arkiv	40%	Schemas claros, atributos tipados, PROJECT_ATTRIBUTE, ownership, expiresIn y relaciones.
Funcionalidad	30%	Crear, leer, filtrar, ver detalle y manejar errores de punta a punta.
Diseno y UX	20%	Interfaz entendible, estados vacios, carga, error y abstraccion

		blockchain.
Codigo y documentacion	10%	README claro, estructura limpia, tipos, variables y pasos reproducibles.

Errores que bajan puntaje

- Guardar un JSON generico sin atributos utiles.
- No usar PROJECT_ATTRIBUTE.
- Consultar solo por entity key.
- Usar strings para numeros que deberian permitir rangos.
- Poner la misma expiracion para todo sin criterio.
- No explicar que datos controla la wallet del usuario.
- Hacer una UI que obliga al usuario a entender blockchain.

10. Plan de 3 dias

Dia	Objetivo	Resultado esperado
Dia 1	Una entidad funcionando	Crear y consultar ai_memory por project + type + userId.
Dia 2	Dos entidades relacionadas + UI	Agregar audit_log, filtros y vista detalle.
Dia 3	Deploy + README + video + pitch	Demo publica, formulario listo y evidencias tecnicas.

11. Checklist de entrega

Producto

- ☐ La app funciona de punta a punta.
- ☐ Hay una demo desplegada y accesible por URL publica.
- ☐ El usuario puede completar el flujo principal sin entender blockchain.
- ☐ La app muestra estados de carga y error.

Arkiv

- ☐ Uso Arkiv como capa de datos principal.
- ☐ Todas las entidades tienen PROJECT_ATTRIBUTE unico.
- ☐ Cada tipo de entidad tiene atributos utiles para consultar.
- ☐ Los numeros estan guardados como numeros, no strings.
- ☐ Hay al menos una query combinada: project + type + otro atributo.
- ☐ expiresIn esta definido con logica de producto.
- ☐ Las relaciones usan atributos compartidos.
- ☐ El README explica el esquema de entidades.

Repo y envio

- ☐ Repo publico.
- ☐ Licencia open source: MIT, Apache 2.0 o equivalente.
- ☐ .env no esta subido.

- [] .env.example si esta subido.
- [] README con instalacion, stack, equipo, demo, video, arquitectura y links.
- [] Video publico de 2 a 3 minutos.
- [] Formulario oficial enviado antes de las 2:00 PM.

12. Template de pitch en espanol

Duracion sugerida: 60 a 90 segundos.

Bloque	Tiempo	Mensaje
Problema	15 s	Las apps de IA guardan memoria y contexto en plataformas cerradas. Eso reduce portabilidad y trazabilidad.
Solucion	20 s	Construimos una app que guarda memorias, auditorias o preferencias como entidades en Arkiv.
Arkiv	25 s	Usamos PROJECT_ATTRIBUTE, atributos tipados, payload JSON, expiracion y relaciones por atributos.
Demo	20 s	Mostramos crear entidad, consultar por atributos, ver payload y explicar metadata.
Cierre	10 s	La memoria de IA pasa a ser portable, auditable y controlada por el usuario.

[NOMBRE] permite que agentes de IA usen memoria controlada por el usuario. Guardamos cada memoria como entidad en Arkiv, con atributos consultables, expiracion y ownership por wallet. Asi, la memoria deja de estar encerrada en una plataforma y se vuelve portable, verificable y util para multiples apps.

13. Troubleshooting

Problema	Revision
PRIVATE_KEY missing	Crear .env y agregar PRIVATE_KEY=0x...
Sin fondos/gas	Usar faucet de Braga con wallet de testnet.
La query no devuelve nada	Verificar PROJECT_SLUG, type, expiracion y tipos de atributos.
Veo datos que no son mios	Falta filtrar por PROJECT_ATTRIBUTE.
Funciona local pero falla en deploy	Revisar variables de entorno, CORS, RPC y no exponer private keys.
Subi una private key a GitHub	Considerarla comprometida y descartar esa wallet.

14. Fuentes oficiales

- Repo oficial: <https://github.com/Arkiv-Network/arkiv-puna-tech-hackathon>
- Formulario de entrega: <https://forms.arkiv.network/punatech26>
- Discord: <https://discord.gg/arkiv>
- PunaTech: <https://www.punatech.ar/>
- Docs Arkiv: <https://docs.arkiv.network>
- Data Explorer: <https://data.arkiv.network/>
- Faucet Braga: <https://braga.hoodi.arkiv.network/faucet/>

Prioridad de fuentes: En caso de conflicto, prevalecen las reglas oficiales del hackathon, el formulario oficial y las instrucciones del equipo de Arkiv por Discord.

15. Comandos para subir el starter a GitHub

```
git init
git add .
git commit -m "Initial Arkiv PunaTech starter"
```

Con GitHub CLI:

```
gh repo create <tu-usuario>/arkiv-punatech-ai-starter --public --source=. --remote=origin --push
```

Sin GitHub CLI:

```
git remote add origin https://github.com/<tu-usuario>/arkiv-punatech-ai-starter.git
git branch -M main
git push -u origin main
```

Entrega recomendada: Antes de enviar el formulario, probar la demo desde una ventana incognito, abrir el repo publico desde otra cuenta, verificar que el video sea publico y guardar capturas de la entidad o transaccion como evidencia.